LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 2

ANDROID LAYOUT WITH COMPOSE

Oleh:

Faisal Tanjung NIM. 2410817310012

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 2

Laporan Praktikum Pemrograman Mobile Modul 2: Android Layout With Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Faisal Tanjung
NIM : 2410817310012

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom NIP. 19971031 202506 1 009

DAFTAR ISI

LEMBAR PENGESAHAN 2
DAFTAR ISI 3
DAFTAR GAMBAR 4
DAFTAR TABEL 5
SOAL 1 6
A. Source Code 8
B. Output Program 18
C. Pembahasan 20
SOAL 2 24
A. Pembahasan 24
DAFTAR PUSTAKA 27
TAUTAN GIT 28

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi 6
Gambar 2. Tampilan Pilihan Persentase Tip 7
Gambar 3. Tampilan Aplikasi Setelah Ditampilkan 7
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose 18
Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose 18
Gambar 6. Screenshot Hasil Jawaban Soal 1 Compose 19
Gambar 7. Screenshot Hasil Jawaban Soal 1 XML 19
Gambar 8. Screenshot Hasil Jawaban Soal 1 XML 20
Gambar 9. Screenshot Hasil Jawaban Soal 1 XML 20

DAFTAR TABEL

Tabel 1. Source Code MainActivity.kt Compose Soal 1 13
Tabel 2. Source Code activity_main.xml XML Soal 1 16
Tabel 3. Source Code MainActivity.kt XML Soal 1 17
Tabel 4. Contoh penggunaan Elvis Operator 26

SOAL 1

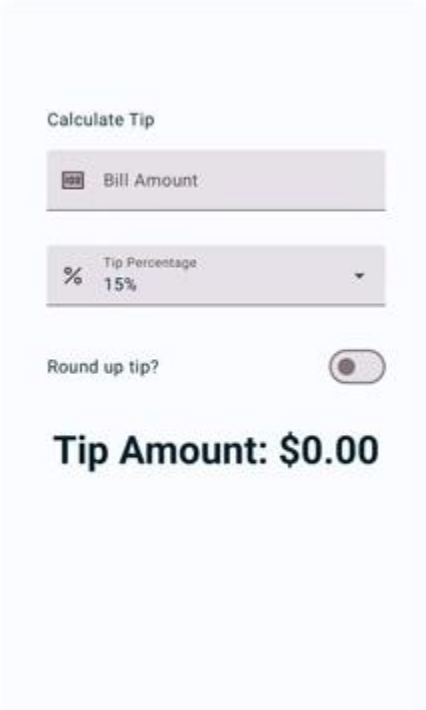
Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

a. Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.

b. Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.

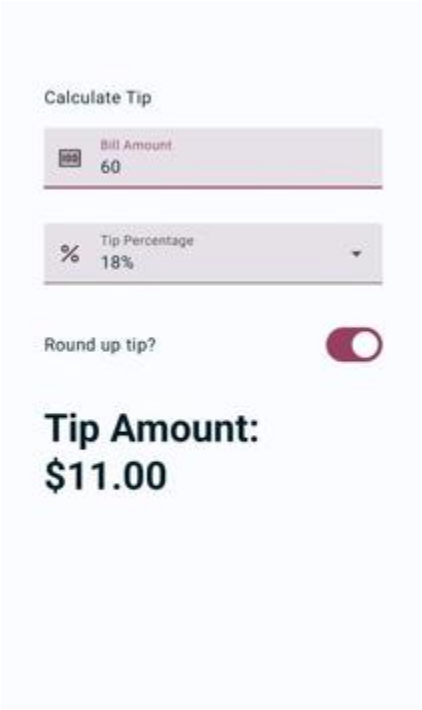
c. Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.

d. Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.



Gambar 1. Tampilan Awal Aplikasi

Gambar 2. Tampilan Pilihan Persentase Tip



Gambar 3. Tampilan Aplikasi Setelah Ditampilkan

A. Source Code

```
package com.example.calculatetip

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import
androidx.compose.foundation.layout.safeDrawingPadding
import
androidx.compose.foundation.layout.statusBarsPadding
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TextField
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import com.example.calculatetip.ui.theme.TipTimeTheme
import java.text.NumberFormat
import androidx.annotation.StringRes
import androidx.compose.ui.text.input.ImeAction
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.size
import androidx.compose.material3.Switch
import
androidx.compose.foundation.layout.wrapContentWidth
import androidx.annotation.DrawableRes
```

```
import androidx.compose.material3.Icon
import androidx.compose.ui.res.painterResource
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Box
import androidx.compose.material3.DropdownMenu
import androidx.compose.material3.DropdownMenuItem
import androidx.compose.material.icons.Icons
import
androidx.compose.material.icons.filled.ArrowDropDown


class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?)
{
        enableEdgeToEdge()
        super.onCreate(savedInstanceState)
        setContent {
            TipTimeTheme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                ) {
                    TipTimeLayout()
                }
            }
        }
    }
}

@Composable
fun TipTimeLayout() {
    var amountInput by remember { mutableStateOf("") }
    var tipInput by remember { mutableStateOf("") }
    var roundUp by remember { mutableStateOf(false) }

    var expanded by remember { mutableStateOf(false) }
    val tipOptions = listOf("15%", "18%", "20%")

    val tipPercent = tipInput.replace("%",
"").toDoubleOrNull() ?: 0.0
    val amount = amountInput.toDoubleOrNull() ?: 0.0
    val tip = calculateTip(amount, tipPercent, roundUp)

    Column(
        modifier = Modifier
            .statusBarsPadding()
            .padding(horizontal = 40.dp)
            .verticalScroll(rememberScrollState())
```

```
            .safeDrawingPadding(),
        horizontalAlignment =
Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text(
            text =
stringResource(R.string.calculate_tip),
            modifier = Modifier
                .padding(bottom = 16.dp, top = 40.dp)
                .align(alignment = Alignment.Start)
        )
        EditNumberField(
            label = R.string.bill_amount,
            leadingIcon = R.drawable.money,
            keyboardOptions =
KeyboardOptions.Default.copy(
                keyboardType = KeyboardType.Number,
imeAction = ImeAction.Next
            ),
            value = amountInput,
            onValueChanged = { amountInput = it },
            modifier = Modifier
                .padding(bottom = 32.dp)
                .fillMaxWidth()
        )
        Box(
            modifier = Modifier
                .padding(bottom = 32.dp)
                .fillMaxWidth()
        ) {
            TextField(
                value = tipInput,
                onValueChange = { },
                readOnly = true,
                label = {
Text(stringResource(R.string.how_was_the_service)) },
                leadingIcon = { Icon(painter =
painterResource(id = R.drawable.percent), null) },
                trailingIcon = {
                    Icon(imageVector =
Icons.Default.ArrowDropDown, contentDescription = null)
                },
                modifier = Modifier.fillMaxWidth()
            )
            Box(
                modifier = Modifier
```

```
                    .matchParentSize()
                    .clickable { expanded = !expanded
})

            DropdownMenu(
                expanded = expanded,
                onDismissRequest = { expanded = false
},
                modifier = Modifier.fillMaxWidth(0.8f)
            ) {
                tipOptions.forEach { option ->
                    DropdownMenuItem(text = {
Text(option) }, onClick = {
                        tipInput = option
                        expanded = false
                    })
                }
            }
        }

        RoundTheTipRow(
            roundUp = roundUp,
            onRoundUpChanged = { roundUp = it },
            modifier = Modifier.padding(bottom = 32.dp)
        )
        Text(
            text = stringResource(R.string.tip_amount,
tip),
            style =
MaterialTheme.typography.displaySmall
        )
        Spacer(modifier = Modifier.height(150.dp))
    }
}

@Composable
fun EditNumberField(
    @StringRes label: Int,
    @DrawableRes leadingIcon: Int,
    keyboardOptions: KeyboardOptions,
    value: String,
    onValueChanged: (String) -> Unit,
    modifier: Modifier = Modifier
) {
    TextField(
        value = value,

```

```
        leadingIcon = { Icon(painter =
painterResource(id = leadingIcon), null) },
        singleLine = true,
        modifier = modifier,
        onValueChange = onValueChanged,
        label = { Text(stringResource(label)) },
        keyboardOptions = keyboardOptions
    )
}

@Composable
fun RoundTheTipRow(
    roundUp: Boolean, onRoundUpChanged: (Boolean) ->
Unit, modifier: Modifier = Modifier
) {

    Row(
        modifier = modifier
            .fillMaxWidth()
            .size(48.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Text(text =
stringResource(R.string.round_up_tip))
        Switch(
            modifier = modifier
                .fillMaxWidth()
                .wrapContentWidth(Alignment.End),
            checked = roundUp,
            onCheckedChange = onRoundUpChanged,
        )
    }
}

private fun calculateTip(amount: Double, tipPercent:
Double = 15.0, roundUp: Boolean): String {
    var tip = tipPercent / 100 * amount
    if (roundUp) {
        tip = kotlin.math.ceil(tip)
    }
    return
NumberFormat.getCurrencyInstance().format(tip)
}

@Preview(showBackground = true)
@Composable
fun TipTimeLayoutPreview() {
```

```
    TipTimeTheme {
        TipTimeLayout()
    }
}

```

Tabel 1. Source Code MainActivity.kt Compose Soal 1

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true">

    <androidx.constraintlayout.widget.ConstraintLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:padding="40dp">

        <TextView
            android:id="@+id/label_calculate_tip"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="20.dp"
            android:text="@string/calculate_tip"
            android:textColor="@color/black"
app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent" />


<com.google.android.material.textfield.TextInputLayout
            android:id="@+id/bill_amount_layout"
style="@style/Widget.MaterialComponents.TextInputLayout.FilledBox"
            android:layout_width="0.dp"
            android:layout_height="wrap_content"
            android:layout_marginTop="16.dp"
app:boxBackgroundColor="@color/field_background_light"
            app:boxStrokeColor="@color/black"
            app:boxStrokeWidth="1.5dp"
            app:boxStrokeWidthFocused="2.5dp"
            app:layout_constraintEnd_toEndOf="parent"
```

```
35
   app:layout_constraintStart_toStartOf="parent"
36
   app:layout_constraintTop_toBottomOf="@id/label_calculate
   _tip"
37             app:startIconDrawable="@drawable/money">
38
39
   <com.google.android.material.textfield.TextInputEditText
40                 android:id="@+id/bill_amount_edit_text"
41                 android:layout_width="match_parent"
42                 android:layout_height="wrap_content"
43                 android:hint="@string/bill_amount"
44                 android:inputType="numberDecimal" />
45
   </com.google.android.material.textfield.TextInputLayout>
46
47
   <com.google.android.material.textfield.TextInputLayout
48             android:id="@+id/tip_percentage_layout"
49
   style="@style/Widget.MaterialComponents.TextInputLayout.
   FilledBox.ExposedDropdownMenu"
50             android:layout_width="0.dp"
51             android:layout_height="wrap_content"
52             android:layout_marginTop="24.dp"
53
   app:boxBackgroundColor="@color/field_background_light"
54             app:boxStrokeColor="@color/black"
55             app:boxStrokeWidth="1.5dp"
56             app:boxStrokeWidthFocused="2.5dp"
57             app:layout_constraintEnd_toEndOf="parent"
58
   app:layout_constraintStart_toStartOf="parent"
59
   app:layout_constraintTop_toBottomOf="@id/bill_amount_lay
   out"
60             app:startIconDrawable="@drawable/percent">
61
62             <AutoCompleteTextView
63
   android:id="@+id/tip_options_auto_complete"
64                 android:layout_width="match_parent"
65                 android:layout_height="wrap_content"
66
   android:hint="@string/how_was_the_service"
67                 android:importantForAccessibility="no"
```

```
                android:inputType="none"
android:textColor="@color/text_color_dark" />
</com.google.android.material.textfield.TextInputLayout>

        <TextView
            android:id="@+id/round_up_label"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/round_up_tip"
app:layout_constraintBottom_toBottomOf="@id/round_up_switch"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toTopOf="@id/round_up_switch" />


<com.google.android.material.materialswitch.MaterialSwitch
            android:id="@+id/round_up_switch"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="24.dp"
            app:layout_constraintEnd_toEndOf="parent"
app:layout_constraintTop_toBottomOf="@id/tip_percentage_layout" />

        <TextView
            android:id="@+id/tip_result"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="32.dp"
android:textAppearance="@style/TextAppearance.MaterialComponents.Headline4"
            android:textStyle="bold"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintTop_toBottomOf="@id/round_up_switch"
            tools:text="Tip Amount: $0.00" />
```

```

    </androidx.constraintlayout.widget.ConstraintLayout>
</ScrollView>
```

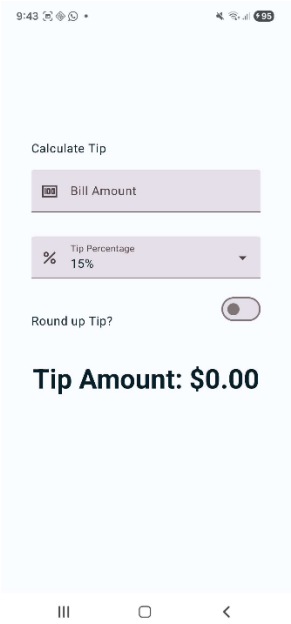
Tabel 2. Source Code activity_main.xml XML Soal 1

```
package com.example.calculatetip

import android.os.Bundle
import android.widget.ArrayAdapter
import androidx.appcompat.app.AppCompatActivity
import androidx.core.widget.doAfterTextChanged
import
com.example.calculatetip.databinding.ActivityMainBindin
g
import java.text.NumberFormat
import java.util.Locale

class MainActivity : AppCompatActivity() {

    private lateinit var binding: ActivityMainBinding
    private var selectedTipPercent: Double = 0.0

    override fun onCreate(savedInstanceState: Bundle?)
{
        super.onCreate(savedInstanceState)
        binding =
ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        displayTip(0.0)

        setupDropdown()

        binding.billAmountEditText.doAfterTextChanged {
            calculateAndDisplayTip()
        }


binding.roundUpSwitch.setOnCheckedChangeListener { _, _
->
            calculateAndDisplayTip()
        }
    }

    private fun setupDropdown() {
        val options = listOf("15%", "18%", "20%")
        val adapter = ArrayAdapter(this,
android.R.layout.simple_list_item_1, options)
```

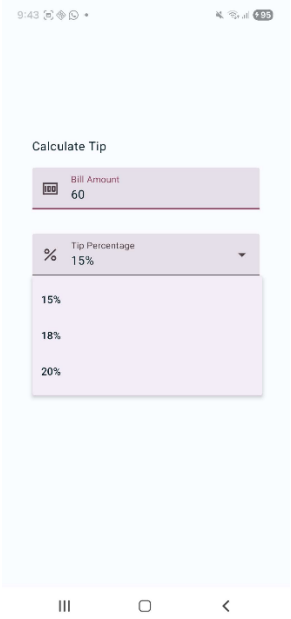
```

binding.tipOptionsAutoComplete.setAdapter(adapter)


binding.tipOptionsAutoComplete.setOnItemClickListener {
_, _, position, _ ->
            val selectedText = options[position]
            selectedTipPercent =
selectedText.replace("%", "").toDoubleOrNull() ?: 0.0
            calculateAndDisplayTip()
        }
    }

    private fun calculateAndDisplayTip() {
        val amountText =
binding.billAmountEditText.text.toString()
        val amount = amountText.toDoubleOrNull() ?: 0.0

        if (amount == 0.0 || selectedTipPercent == 0.0)
{
            displayTip(0.0)
            return
        }

        var tip = (selectedTipPercent / 100) * amount

        if (binding.roundUpSwitch.isChecked) {
            tip = kotlin.math.ceil(tip)
        }

        displayTip(tip)
    }

    private fun displayTip(tip: Double) {
        val formattedTip =
NumberFormat.getCurrencyInstance(Locale.US).format(tip)
        binding.tipResult.text =
getString(R.string.tip_amount, formattedTip)
    }
}

```

Tabel 3. Source Code MainActivity.kt XML Soal 1

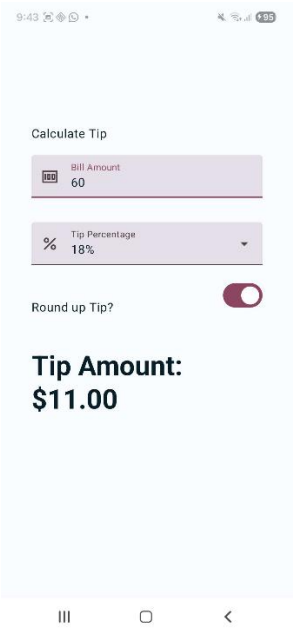
B. Output Program



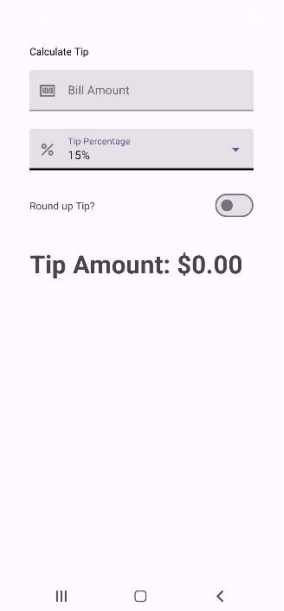
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose



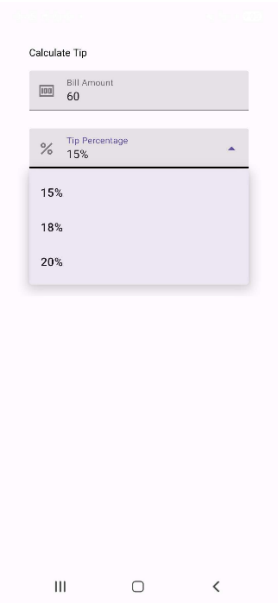
Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose



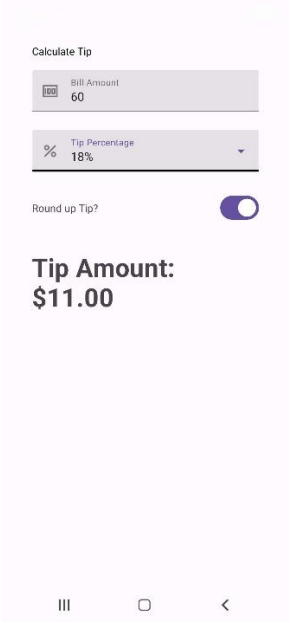
Gambar 6. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 7. Screenshot Hasil Jawaban Soal 1 XML



Gambar 8. Screenshot Hasil Jawaban Soal 1 XML



Gambar 9. Screenshot Hasil Jawaban Soal 1 XML

C. Pembahasan

File ‘MainActivity.kt’ **Compose** merupakan aplikasi penghitung tip yang mengimplementasikan UI Toolkit Jetpack Compose untuk mengelola input pengguna secara dinamis dan menampilkan hasil perhitungan secara real-time. Fungsi ‘onCreate()’ bertindak sebagai pintu masuk utama yang dipanggil sistem Android saat aplikasi pertama kali dijalankan, di mana di dalamnya terdapat ‘setContent{}’ yang berfungsi untuk memasang

komponen UI ke dalam activity serta ‘TipTimeTheme’ untuk menerapkan tema warna dan tipografi pada aplikasi. Fungsi utama ‘TipTimeLayout()’ berperan sebagai pusat pengelolaan struktur tampilan dan status (state) aplikasi, yang menyimpan variabel ‘amountInput’, ‘tipInput’, dan ‘roundUp’ menggunakan fungsi ‘remember’ dan ‘mutableStateOf()’ agar nilai yang dimasukkan pengguna tetap tersimpan di memori dan tidak hilang saat terjadi rekomposisi atau pembaruan layar. Di dalam fungsi ini, terdapat penggunaan Box yang dikombinasikan dengan ‘TextField’ dan ‘DropdownMenu’ untuk menciptakan menu pilihan persentase tip yang interaktif, di mana variabel expanded bertugas mengontrol muncul atau tidaknya daftar pilihan tersebut. Selanjutnya, terdapat fungsi ‘EditNumberField()’ yang merupakan komponen input teks khusus yang menerima parameter seperti ‘label’, ‘leadingIcon’, dan ‘keyboardOptions’ untuk mempermudah pengguna memasukkan angka tagihan dengan keyboard numerik. Fungsi ‘RoundTheTipRow()’ digunakan untuk membangun baris yang berisi komponen Switch, yang memungkinkan pengguna memilih untuk membulatkan hasil perhitungan tip ke atas atau membiarkannya sesuai angka desimal. Untuk bagian logika bisnis, terdapat fungsi privat ‘calculateTip()’ yang melakukan perhitungan matematis antara jumlah tagihan dan persentase tip, serta menggunakan ‘NumberFormat’ untuk mengubah angka mentah tersebut menjadi format mata uang yang rapi. Terakhir, fungsi ‘TipTimeLayoutPreview()’ yang dilengkapi dengan anotasi ‘@Preview’ memungkinkan developer untuk melihat visualisasi UI secara langsung di panel desain Android Studio tanpa harus melakukan running pada emulator, sehingga proses pengembangan menjadi jauh lebih efisien.

File ‘MainActivity.kt’ **XML** ini merupakan aplikasi penghitung tip yang mengimplementasikan pendekatan View Binding untuk menghubungkan logika kode dengan elemen antarmuka secara aman tanpa perlu menggunakan ‘findViewById()’. Fungsi ‘onCreate()’ bertindak sebagai pintu masuk utama aplikasi yang menginisialisasi variabel binding melalui ‘ActivityMainBinding.inflate(layoutInflater)’ dan memasang akar tampilan ke layar melalui ‘setContentView(binding.root)’, serta memastikan tampilan awal dimulai dengan nilai nol melalui fungsi ‘displayTip(0.0)’. Logika interaksi pengguna dimulai pada fungsi ‘setupDropdown()’ yang menggunakan ArrayAdapter untuk mengisi komponen ‘tipOptionsAutoComplete’ dengan pilihan persentase 15%, 18%, dan 20%, di mana listener ‘setOnItemClickListener’ bertugas menangkap pilihan pengguna dan mengubah teks tersebut

menjadi nilai angka untuk disimpan dalam variabel ‘selectedTipPercent’. Untuk menciptakan pengalaman pengguna yang responsif, aplikasi menggunakan listener ‘doAfterTextChanged’ pada kolom input tagihan dan ‘setOnCheckedChangeListener’ pada tombol switch pembulatan, sehingga fungsi ‘calculateAndDisplayTip()’ akan dipanggil secara otomatis setiap kali ada perubahan teks atau status tombol tanpa mengharuskan pengguna menekan tombol hitung secara manual. Di dalam fungsi ‘calculateAndDisplayTip()’, sistem melakukan validasi data dengan mengonversi input teks menjadi angka desimal melalui ‘toDoubleOrNull()’, melakukan perhitungan perkalian persentase, dan secara cerdas menerapkan logika pembulatan ke atas menggunakan fungsi ‘kotlin.math.ceil()’ jika fitur ‘roundUpSwitch’ sedang aktif.

Terakhir, yaitu file layout ‘activity_main.xml’ **XML** yang dirancang menggunakan arsitektur View klasik berbasis XML yang diawali dengan komponen ‘ScrollView’ sebagai elemen akar guna memastikan seluruh konten tetap dapat diakses melalui pengguliran layar apabila resolusi perangkat terbatas, dengan atribut ‘android:fillViewport="true"’ yang memungkinkan kontainer di dalamnya tetap mengisi seluruh ruang vertikal yang tersedia. Di dalam ‘ScrollView’ terdapat ‘ConstraintLayout’ sebagai wadah utama yang memberikan fleksibilitas tinggi dalam memposisikan elemen antarmuka melalui sistem pengait (constraints) serta memiliki padding sebesar ‘40dp’ untuk memberikan ruang kosong yang proporsional di sekeliling layar. Komponen visual dimulai dengan ‘TextView’ untuk label instruksi, diikuti oleh ‘TextInputLayout’ bermodel ‘FilledBox’ yang berfungsi sebagai pembungkus dekoratif bagi ‘TextInputEditText’, di mana penggunaan atribut ‘inputType="numberDecimal"’ secara spesifik memerintahkan sistem untuk hanya menampilkan keyboard angka desimal saat pengguna memasukkan nominal tagihan. Selanjutnya, terdapat ‘TextInputLayout’ kedua dengan gaya ‘ExposedDropdownMenu’ yang dipadukan dengan ‘AutoCompleteTextView’ untuk menciptakan antarmuka pemilihan persentase tip yang elegan, di mana atribut ‘startIconDrawable’ memberikan petunjuk visual berupa ikon persen. Untuk pengaturan logika tambahan, terdapat ‘MaterialSwitch’ yang diposisikan secara horizontal sejajar dengan ‘TextView’ label pembulatan melalui constraint dasar guna memungkinkan pengguna mengaktifkan atau menonaktifkan fitur pembulatan tip ke atas secara interaktif. Terakhir, elemen ‘TextView’ dengan ID ‘tip_result’ bertindak sebagai area tampilan output utama yang menggunakan gaya textAppearance Headline4 dan

teks tebal (bold) untuk menonjolkan hasil perhitungan tip akhir agar informasi biaya yang harus dibayarkan dapat terlihat sangat jelas oleh pengguna.

SOAL 2

Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.

Additional:

Apa saja fitur yang menjadi daya jual pada Kotlin? (contoh: null safety, chain, dll)**

berikan 1 test case terkait penggunaan fitur unggulan yang ada pada kotlin (contoh: penggunaan null safety)

A. Pembahasan

Perbedaan mendasar antara Jetpack Compose dan XML terletak pada pergeseran paradigma dari metode imperatif yang tradisional ke arah deklaratif yang lebih modern. Dalam sistem XML, kita sering kali terjebak dalam proses pengelolaan elemen UI secara manual yang cukup melelahkan, di mana setiap perubahan data menuntut instruksi eksplisit melalui View Binding atau findViewById untuk memperbarui tampilan. Hal ini menciptakan beban kerja ekstra dan risiko ketidakkonsistenan status (state) karena adanya pemisahan antara file tata letak dan logika bisnis. Sebaliknya, Jetpack Compose memungkinkan kita untuk membangun antarmuka secara langsung di dalam Kotlin dengan mendeskripsikan bagaimana UI seharusnya terlihat pada kondisi tertentu; sistem akan secara otomatis melakukan sinkronisasi dan memperbarui layar setiap kali terjadi perubahan data melalui mekanisme recomposition, yang secara drastis memangkas jumlah kode repetitif dan meminimalkan potensi kesalahan logika.

Jetpack Compose memberikan keuntungan besar dalam hal efisiensi dan modularitas kode, terutama berkat kemampuannya untuk membuat komponen yang dapat digunakan kembali secara fleksibel tanpa kerumitan atribut XML yang kaku. Dengan fitur Live Preview yang interaktif, kita bisa melihat perubahan desain secara instan, yang sangat mempercepat siklus pengembangan aplikasi yang kompleks. Penggunaan satu bahasa pemrograman tunggal (Full Kotlin) juga membuat integrasi antara logika aplikasi dan elemen visual menjadi jauh lebih mulus, termasuk saat menangani animasi yang dinamis. Namun, perlu diingat bahwa Compose memiliki kurva pembelajaran yang cukup menantang bagi mereka yang sudah terbiasa dengan struktur View klasik, serta adanya tantangan berupa waktu

kompilasi yang mungkin terasa sedikit lebih berat pada proyek skala besar atau pada perangkat dengan spesifikasi terbatas karena proses transformasi kode yang intensif.

Di sisi lain, implementasi berbasis XML tetap memegang keunggulan dalam hal kematangan ekosistem dan stabilitas karena telah menjadi standar utama selama bertahun-tahun di industri Android. Pendekatan ini menawarkan pemisahan tanggung jawab (separation of concerns) yang sangat tegas antara desain visual dan logika kontrol, yang memudahkan kolaborasi antar tim desainer dan pengembang. Dukungan dokumentasi yang melimpah serta pustaka pihak ketiga yang sudah sangat mapan menjadikannya pilihan yang sangat aman untuk aplikasi yang memerlukan kompatibilitas luas dengan perangkat versi lama. Meskipun begitu, XML sering kali terasa kurang gesit dalam menangani antarmuka yang sangat dinamis, karena sifatnya yang statis memaksa pengembang untuk menulis banyak kode "jembatan" yang cenderung membuat struktur proyek menjadi lebih gemuk dan sulit dipelihara dibandingkan dengan kesederhanaan yang ditawarkan oleh pendekatan deklaratif.

Additional

Ada banyak fitur yang menjadi daya jual bagi bahasa pemrograman Kotlin, Tetapi saya akan lebih spesifik ke dua fiturnya, yaitu fitur Null Safety dan Fitur Kotlin Multiplatform (KMP). Null Safety merupakan fitur yang terkenal dengan keamanan NULL nya untuk membantu pengembang mencegah pengecualian penunjuk NULL yang dimana itu merupakan sumber kesalahan runtime di bahasa pemrograman Java. Kotlin mengharuskan penanganan eksplisit nilai NULL dengan memperkenalkan tipe dan operator nullable nya, seperti ?, ?:, dan !!. Jadi dengan operator – operator ini, kita dapat memberitahu kotlin bahwa variabel atau apapun itu yang ada tipe datanya dapat bernilai NULL selama kita menaruh operator tersebut. (Suresh, Suresh, Prashant, Ahemad, & Arun, 2025)

Kotlin Multiplatform (KMP) merupakan fitur yang dapat membuat kita melakukan implementasi code ke platform lain. KMP menggunakan framework Compose Multiplatform untuk membuat UI. Dengan framework ini, pengembang dapat mengimplementasikan UI sekali dan menggunakannya kembali ke banyak platform yang pengembang targetkan, seperti iOS, Android, Desktop, dan lain – lain. Dengan menggabungkan Compose dan KMP,

pengembang dapat mencapai codebase dari kotlin sekitar 80% - 95% tergantung kompleksitasnya. (Stanić & Ćirković, 2024)

Fitur Null Safety pada Bahasa Kotlin bertujuan untuk melakukan validasi ketahanan suatu aplikasi dalam menangani data kosong, seperti misalnya pada aplikasi CRUD yang saya buat baru – baru ini, disitu user dapat menginputkan dua hal, yaitu id akun dan nickname akun (dalam konteks akun mobile legends). Jadi pada program CRUD ini, saya membuat fitur pengisian nickname ini dapat dikosongkan (null) karena menurut saya dalam mencari sebuah akun itu, cukup mengetahui id nya saja maka akunnya akan ketemu dan menurut saya tidak semua orang serajin itu untuk menulis nicknamenya juga jadi saya buat fitur ini agar dapat dikosongkan menggunakan elvis operator (?:). Dengan begini, Kotlin tidak akan menghentikan programnya karena kita sudah memberi tanda bahwa data ini boleh dikosongkan.

```
    println("\n---Show Data (Key, Value)---")
        for (akun in listAkun) {
            val namaTampil = akun.nickName ?: "NULL"

            val formatData = DataClass(key = akun.id, value
= namaTampil)
            println("Key:   ${formatData.key}   -->   Value:
${formatData.value}")
        }
```

Tabel 4. Contoh penggunaan Elvis Operator

DAFTAR PUSTAKA

Stanić, N., & Ćirković, S. (2024). Analysis of Approaches to Developing Kotlin Multiplatform Applications and Their Impact on Software Engineering. *10th International Scientific Conference Technics, Informatics, and Education*, 53-59.

Suresh, B. R., Suresh, B. R., Prashant, K. P., Ahemad, S. A., & Arun, G. L. (2025). The Study of Kotlin Programming Language Like Features, Applications, and Advancements. *International Journal of Advanced Research in Science, Communication and Technology*, 592-594.

TAUTAN GIT

Link git: https://github.com/Lawdiy/MODUL-MOBILE/tree/7e006ca1fd52cf0ae428146cc7f69b91434c0e3d/Modul%202